VERILOG · TEAM PROJECT 02

UART Loopback

설계 · 시뮬레이션 검증 · 보드 시연



0x30 '0' · 8N1 · 9600 bps

정안성

2026. 10

clk

100 MHz

Basys-3

baud

9600 bps

8N1 · LSB first

oversampling

16x

b_tick 6.51 μs

sampling

ONE BIT mode

bit 중앙 1회

Contents

PC → uart_rx → uart_tx → PC loopback으로
RX · TX를 한 번에 검증

00 **Summary** — 핵심 수치 · 검증 결과 — p. 3

01 **목적** — 설계 대상 · 목표 동작 — p. 4

02 **UART Reference** — 프로토콜 · 16x oversampling · margin · 16x 선택 이유 — p. 5 – 8

03 **블록 다이어그램** — uart_loopback · top_ascii — p. 9 – 10

04 **ASM 차트** — uart_rx · uart_tx — p. 11 – 12

05 **시뮬레이션** — 시나리오 · RX 타이밍 · Data sampling · TX echo — p. 13 – 16

06 **트러블 슈팅** — 현상 → 원인 → 해결 → 적용 결과 — p. 17 – 18

07 **시연** — ComPortMaster · Basys-3 FND — p. 19

08 **결론 · 고찰** — 결론 · ONE BIT mode 한계 · 개선 방향 — p. 20

A **APPENDIX** — Glitch 입력 방법 · 폭 기준 · 결과 — p. 22 – 26

00

16x Oversampling UART Loopback 구현 · 검증 결과

결론

- **uart_rx 출력을 uart_tx 입력에 직결 → PC 송신 byte를 그대로 echo · FND 4자리 표시**
- **256 byte echo · START glitch 23 case 모두 PASS, data bit glitch는 bit 중앙 64 case FAIL (ONE BIT mode)**

Oversampling

16x

b_tick 6.51 μs · 1 bit = 16 tick

동작 주파수

100 MHz

÷ 651 → 153.6 kHz b_tick → 9600.6 bps

Sampling margin

43.75 %

start 검출 지연 1 tick 반영 (늦은 쪽)

Loopback echo

256 / 256

0x00 ~ 0xFF 송신 = echo

START glitch

23 / 23

수정 전 6 / 23 → 수정 후 전부 PASS

Data bit glitch

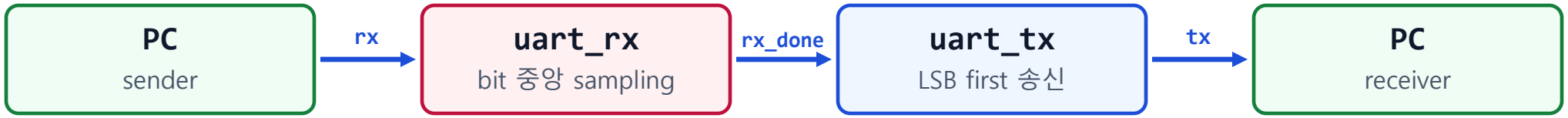
256 / 320

bit 중앙 sampling edge 64 case FAIL

01

PC ↔ FPGA UART 송수신 구현 · Loopback으로 RX · TX 동시 검증

결론 · 받은 byte를 그대로 되돌려 보내는 loopback → echo 하나로 uart_rx · uart_tx 둘 다 확인
· 수신 문자를 FND에 표시 · glitch가 들어와도 가짜 byte를 받지 않는 RX



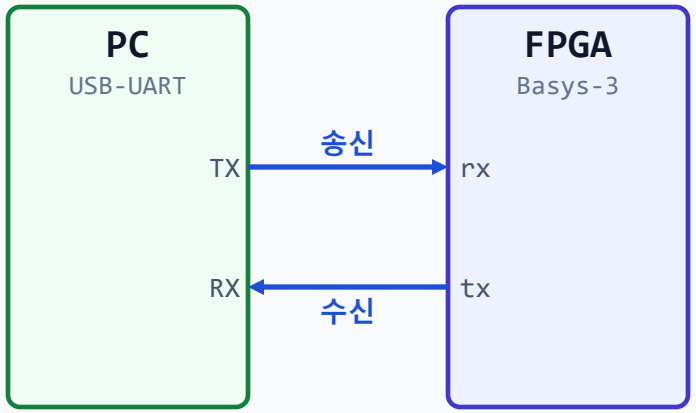
#	대상	목표 동작	검증
1	uart_rx	rx fall → START 8 tick으로 bit 중앙 정렬 → 16 tick마다 1 bit → rx_done	시나리오 #1 · #2
2	uart_tx	tx_start → START · D0 ~ D7 (LSB first) · STOP 각 16 tick	시나리오 #3
3	uart_loopback	rx_done → tx_start, rx_data → tx_data 직결 → echo	시나리오 #3 · 256 case
4	top_ascii	ASCII → hex → FND 4자리 shift, hex 외 문자 '-'	보드 시연
5	glitch 내성	START 구간 glitch → IDLE 복귀, 가짜 byte 없음	시나리오 #4 · #5

02

UART: Clock 없이 Start Bit로 시점을 맞추는 비동기 직렬 통신

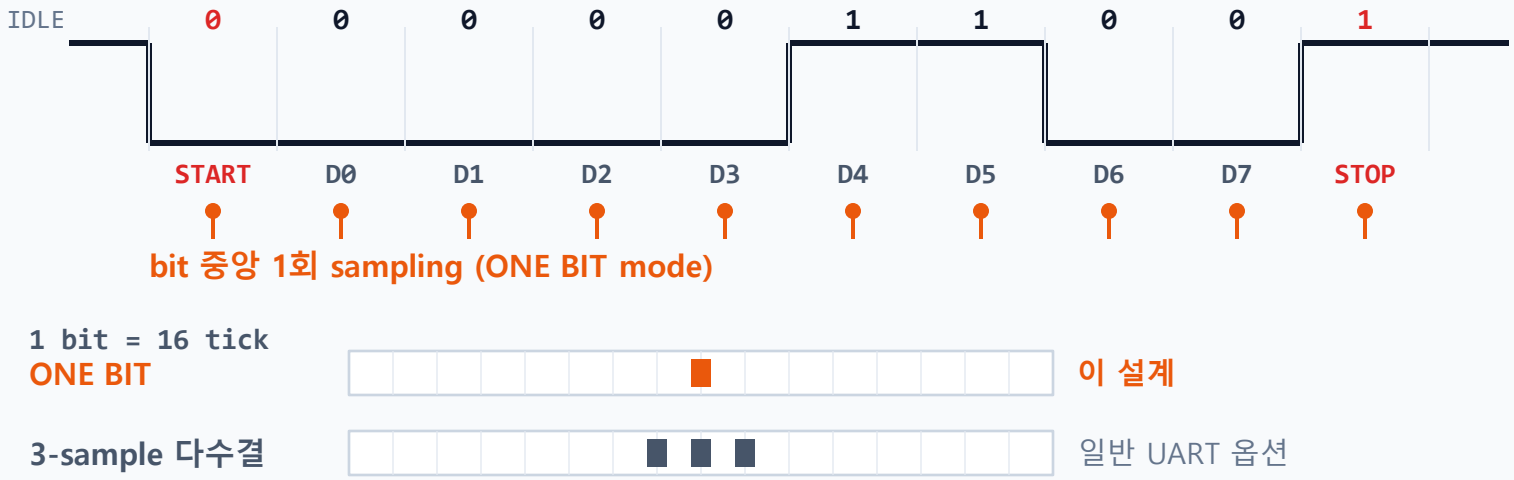
결론 · TX · RX 선이 따로 → 송신 · 수신 동시 가능 (full-duplex), 공유 clock 없음 → 양쪽이 baud만 약속
· 8N1 frame = START(0) → D0 ~ D7 (LSB first) → STOP(1), 이 설계는 bit 중앙 1회 sampling (ONE BIT mode)

연결



선 2개 → 동시 송수신

8N1 frame · '0' = 0x30



1 **Start bit** 1 → 0 falling edge = frame 시작
이 edge로 bit 경계 재정렬

2 **Stop bit** 항상 1 → 다음 falling edge 보장
오차가 frame마다 초기화

3 **LSB first** D0부터 송신 → 0x30은 0,0,0,0,1,1,0,0
RX는 MSB에 넣고 오른쪽 shift

4 **ONE BIT mode** bit 중앙 sample 1개로 판정
bit 중앙 glitch에 취약 (부록)

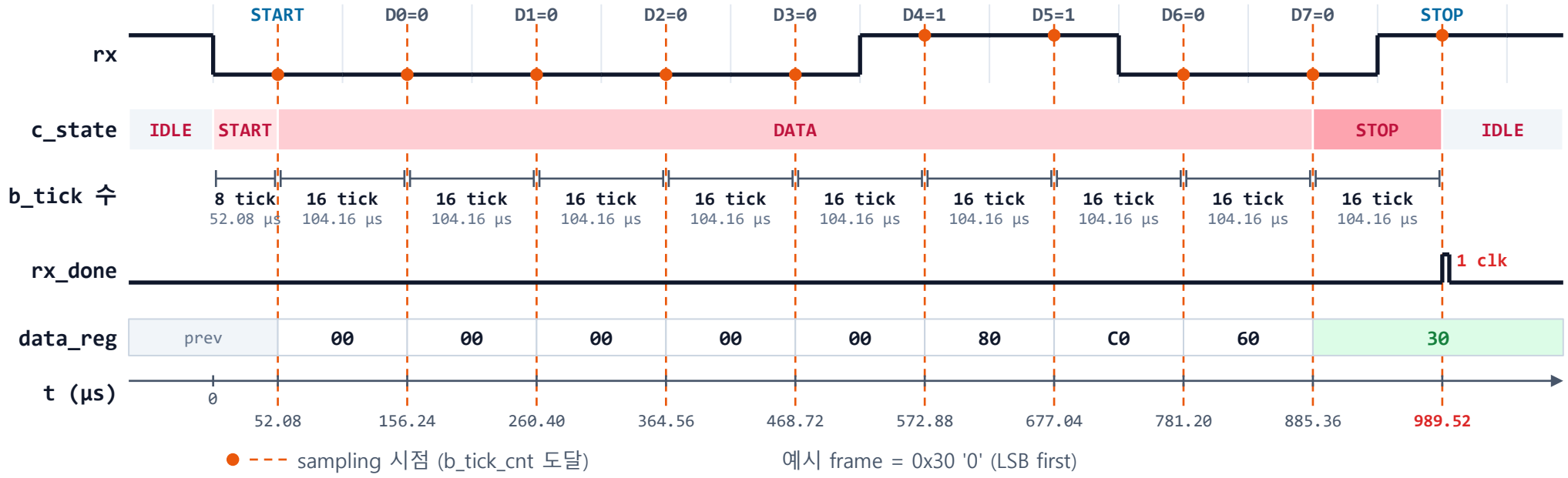
02

100 MHz ÷ 651 = b_tick, 16 tick = 1 bit (104.16 µs)

결론

- F_COUNT = 100 MHz ÷ (9600 × 16) = 651 → b_tick 6.51 µs, 1 bit = 16 tick = 104.16 µs (9600.6 bps)
- START 8 tick 대기로 bit 중앙 정렬 → 이후 16 tick마다 sampling, rx_done은 STOP 중앙

F_COUNT	b_tick 주기	1 bit	start 중앙
100 MHz ÷ 153,600 = 651	651 × 10 ns = 6.51 µs	16 tick = 104.16 µs	START 8 tick = 52.08 µs



xsim 측정 (tb_measure.sv)

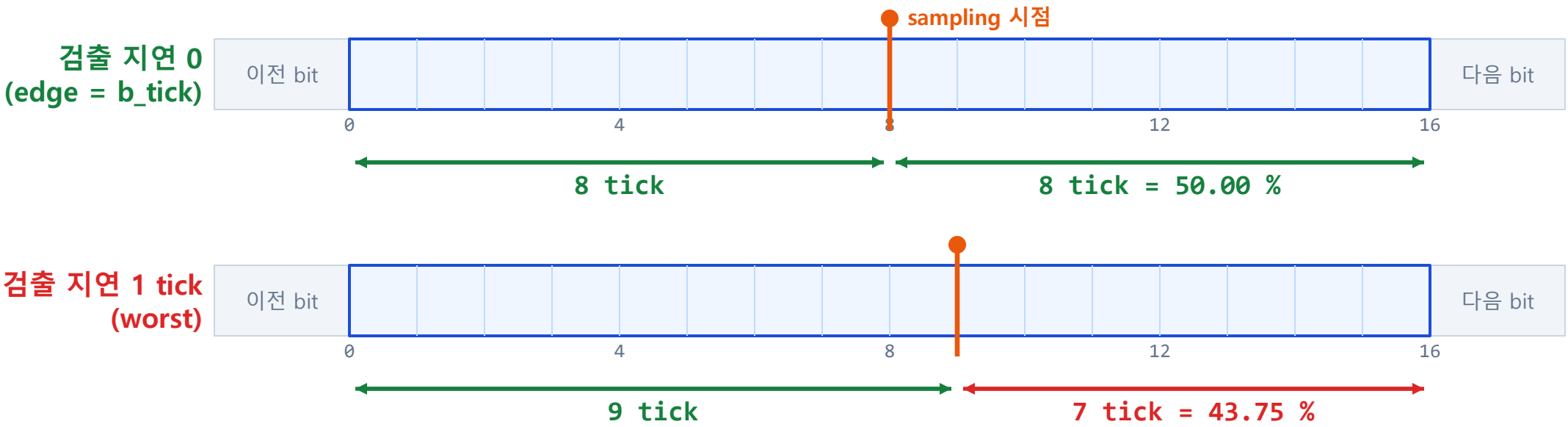
b_tick 주기	**6.51 µs**	START 체류	**52.08 µs**	sample 간격	**104.16 µs**
첫 sample	**rx fall + 156.245 µs**	rx_done	**rx fall + 989.525 µs**	TX 시작	**rx_done + 10 ns**

02

Sampling Margin: Start 검출 지연 최대 1 tick → 늦은 쪽 43.75 %

결론

- **Margin = sampling 시점에서 bit 경계까지 남은 거리, bit 중앙 sampling = 양쪽 8 tick (50 %)**
- **rx fall은 b_tick 순간에만 검출 → 0 ~ 1 tick 늦게 START → 늦은 쪽 7 tick = 43.75 %**



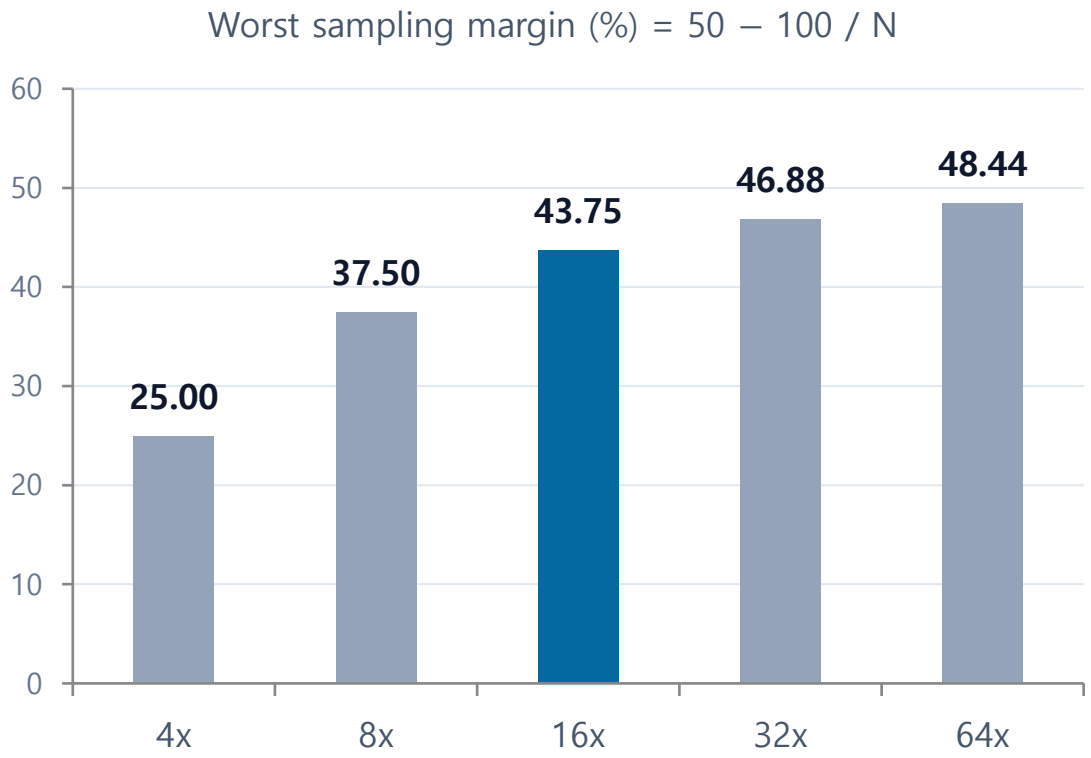
1. **지연 원인** b_tick = 1 순간만 rx 확인 → 최대 1 tick 늦게 START
2. **늦은 쪽** sample이 오른쪽으로 밀림 → 다음 bit까지 7 tick (43.75 %)
3. **이른 쪽** 왼쪽 여유는 8 ~ 9 tick → 최소 50 % 유지
4. **누적 X** START마다 falling edge로 다시 정렬 → 지연은 frame마다 초기화, xsim에서 모든 frame rx fall → rx_done 989.525 µs 동일

02

16x: Margin · Baud 정확도 · 최대 속도의 균형점

결론

- **Oversampling N을 키워도 worst margin = 50 − 100/N %로 조금씩만 증가 (16x 43.75 % → 32x 46.88 %)**
- **N이 클수록 F_COUNT 정수 내림 오차 ↑ (32x +0.16 %) · 같은 clk에서 최대 baud = clk ÷ N ↓**



F_COUNT = 100 MHz ÷ (9600 × N), Verilog 정수 나눗셈 내림

N	F_COUNT	실제 baud	오차	최대 baud
4x	2604.17 → 2604	9600.61	+0.0064 %	25.00 Mbps
8x	1302.08 → 1302	9600.61	+0.0064 %	12.50 Mbps
16x	**651.04 → 651**	**9600.61**	**+0.0064 %**	**6.25 Mbps**
32x	325.52 → 325	9615.38	+0.16 %	3.125 Mbps
64x	162.76 → 162	9645.06	+0.47 %	1.563 Mbps

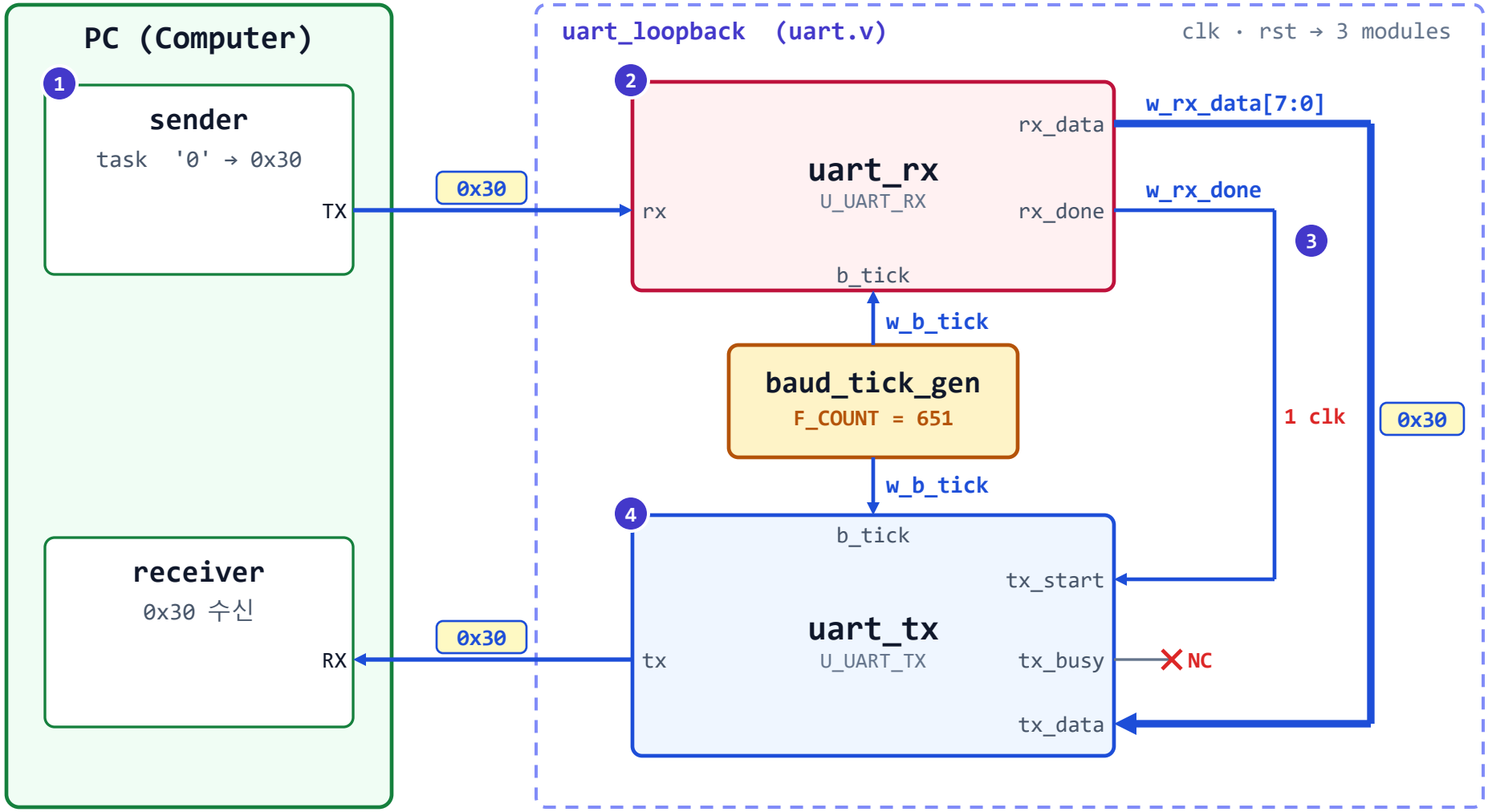
1 **16x** margin 43.75 % · 오차 +0.0064 % · 최대 6.25 Mbps
→ 9600 bps에 충분, 업계 표준 배수

2 **더 키울 경우** 32x: margin +3.1 %p 대신 baud 오차 25배 · 최대 속도 절반 → 노이즈 대책은 N보다 sample 다수결

RX 출력이 TX 입력으로 직결된 Loopback 구조

03

결론 · PC sender가 보낸 '0'(0x30)을 uart_rx가 복원 → rx_data · rx_done이 uart_tx의 tx_data · tx_start로 직결
· baud_tick_gen 하나를 RX · TX가 공유 (b_tick 6.51 μs) → PC receiver로 0x30 그대로 echo



1 sender

'0' → 0x30
frame 송신

2 uart_rx

bit 중앙 sampling
→ rx_data 0x30

3 rx_done

STOP 중앙 1 clk
= tx_start

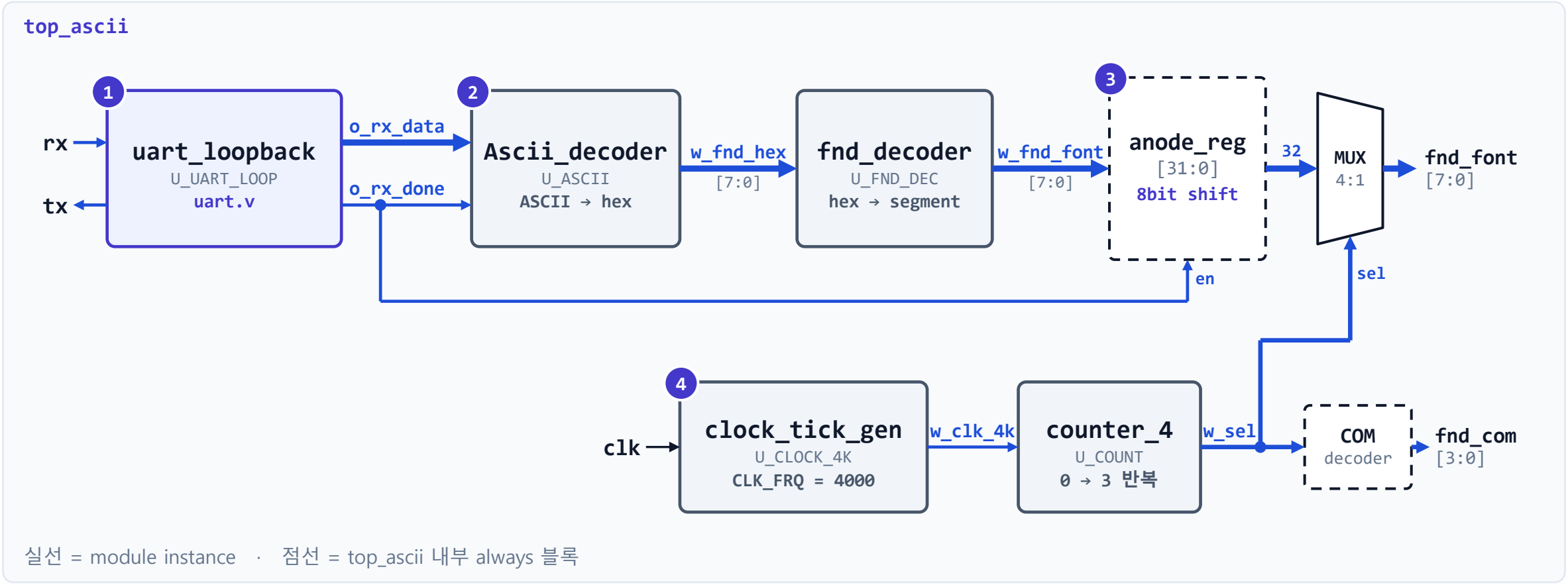
4 uart_tx

0x30 LSB first
→ receiver

top_ascii: rx_done마다 수신 문자를 FND 4자리로 Shift

03

결론 · rx_done마다 ASCII → hex → 7-seg font 변환 후 anode_reg 8 bit shift → 최근 4글자 표시
· hex 외 문자 (CR · LF 등)는 8'hFF → fnd_decoder default '-', reset 값도 '----'



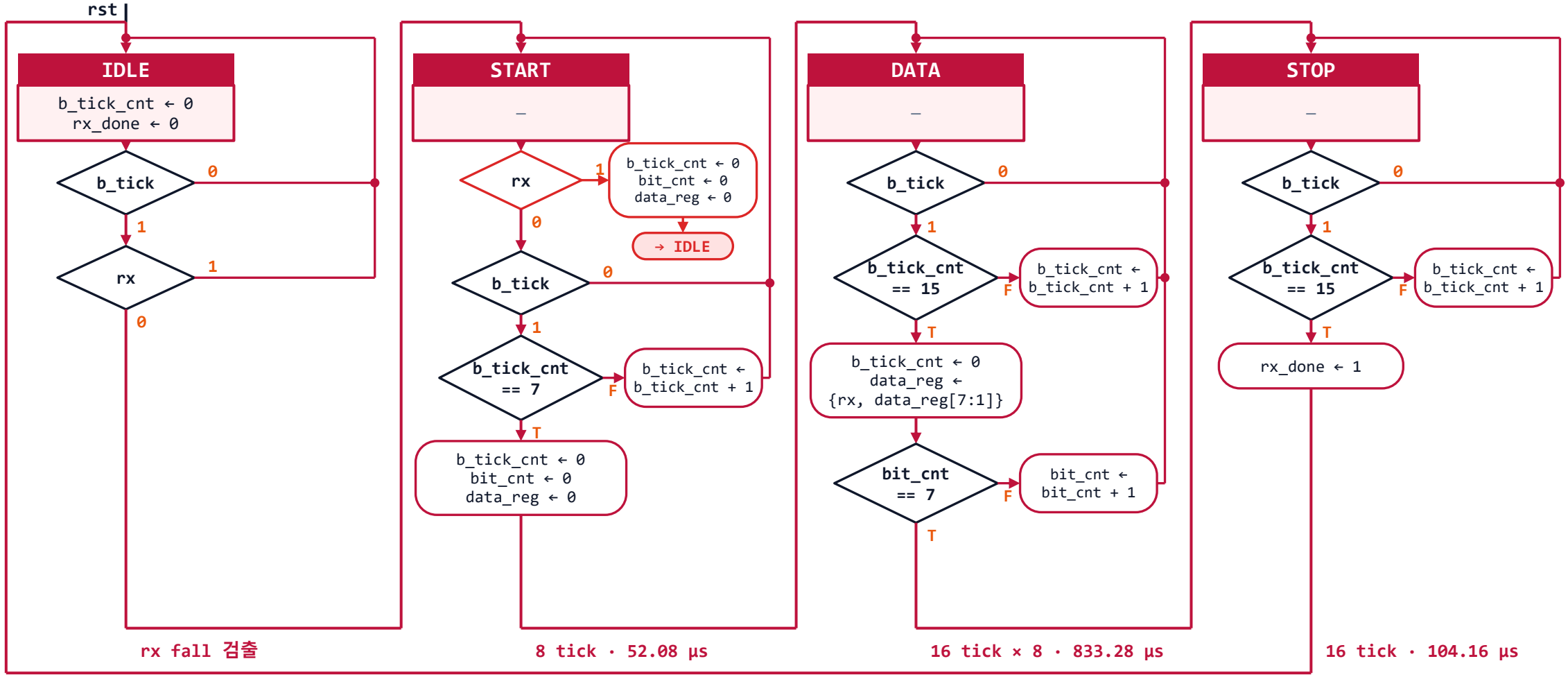
1 **echo**	rx_data · rx_done	
2 **decode**	hex 외 → 8'hFF	
3 **shift**	rx_done마다 8 bit	
4 **scan**	4 kHz digit 선택	

uart_rx ASM: START 중앙 정렬 후 16 tick마다 Sampling

04

결론

- rx fall 검출 → START 8 tick 동안 rx = 0 유지 확인, rx = 1일 경우 IDLE 복귀 (glitch 거름) → bit 중앙 도달
- DATA에서 16 tick마다 1 bit shift (8회) → STOP 중앙에서 rx_done 1 clk

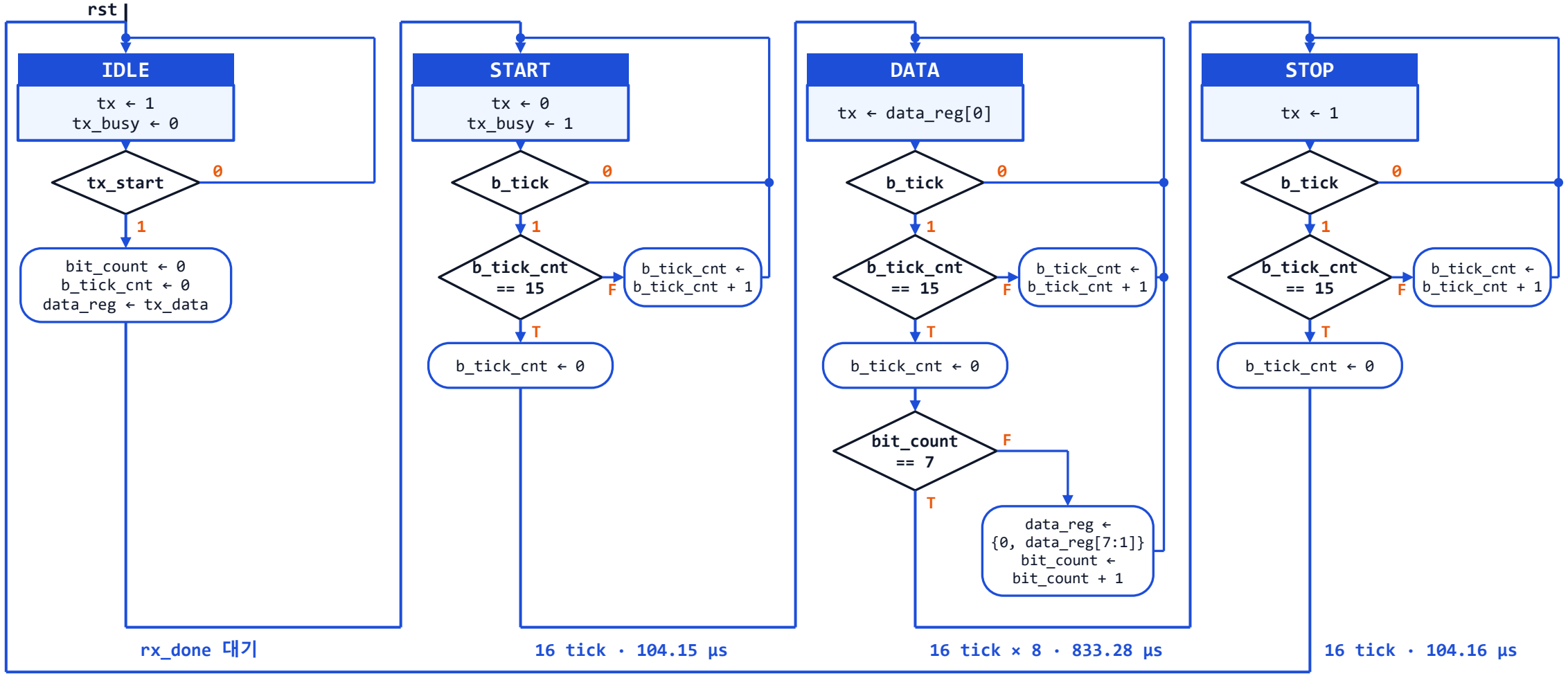


04

uart_tx ASM: tx_start 후 각 Bit 16 tick 유지, LSB First

결론

- IDLE에서 tx_start = 1 → tx_data latch → START(0) · D0 ~ D7 · STOP(1) 각 16 tick 송신
- tx_start는 IDLE에서만 확인, tx_busy는 START ~ STOP 동안 1



UART_LOOPBACK 검증 시나리오

05

결론 · 시나리오 5개 — RX 타이밍 · Data sampling · TX echo (256 case) · START glitch (23 case) · Data bit glitch (320 case)
· #1 ~ #4 기대값 일치, #5는 bit 중앙 glitch 64 case FAIL → ONE BIT mode 한계 (고찰)

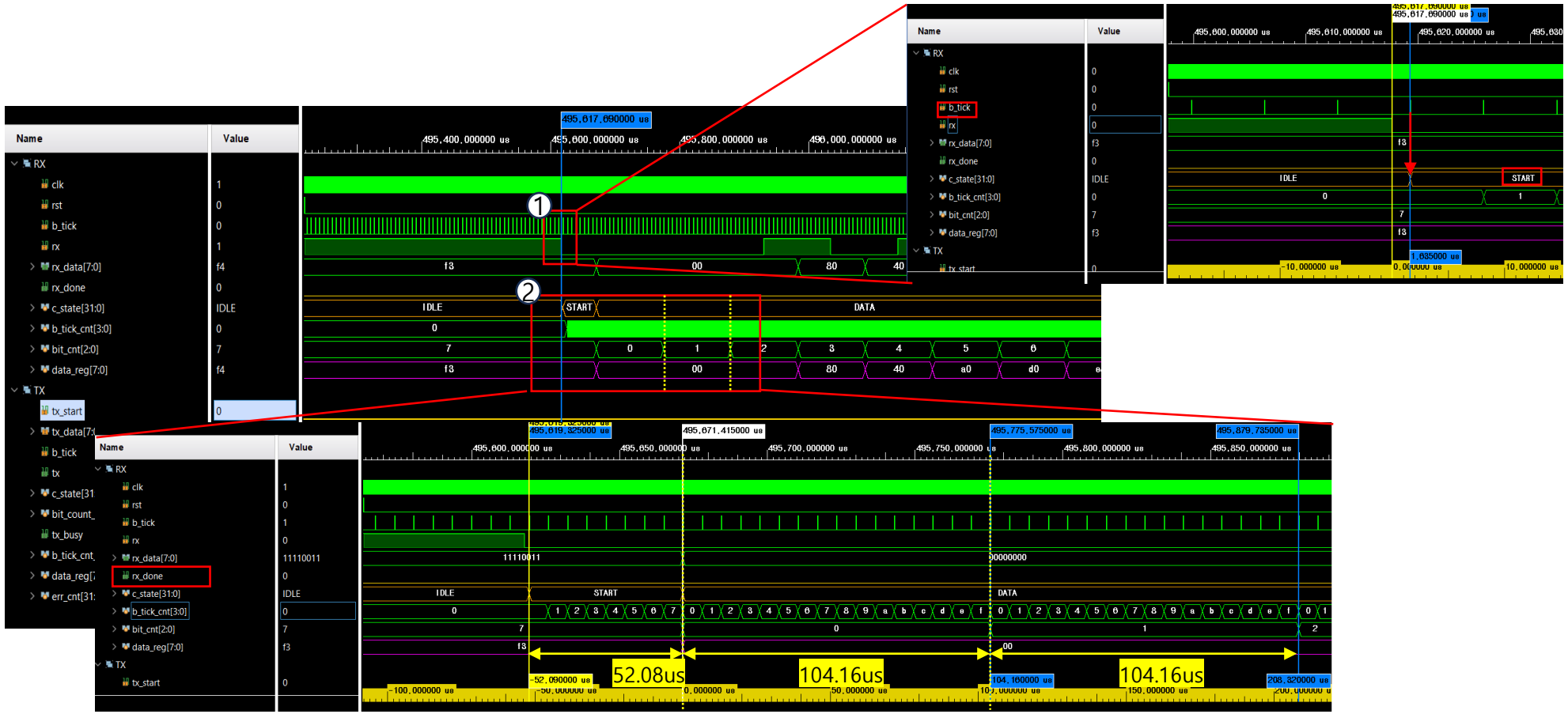
#	시나리오	입력 (Stimulus)	확인 신호	예상 결과	결과
1	**rx 프로토콜 동작 타이밍 분석** p. 14	rx 1 → 0 후 9600 bps 간격 데이터 송신 (sender), stop 0 → 1	rx, c_state, b_tick_cnt, state 간 타이밍	rx 1 → 0 후 다음 b_tick에서 START, b_tick_cnt == 7에서 DATA	일치
2	**Data sampling** p. 15	8'h00 ~ 8'hFF 256 case input (rx sender)	rx, rx_data_reg, tx, tx_data_reg	DATA 구간 b_tick_cnt == 15마다 rx_data shift, LSB부터 채움	일치
3	**TX 송신 · Loopback echo** p. 16	#2와 같은 256 byte, receive_uart로 tx 역변환 후 비교	rx_done, tx_start, tx, tx c_state	rx_done → tx_start, START · D0 ~ D7 · STOP, echo = 송신 byte	256 / 256
4	**START glitch** p. 17 · 부록	20 ns ~ 50 µs low pulse 16개 · burst 2개 · glitch 뒤 byte 5개	rx_done 횟수, tx echo 횟수, rx_data	glitch 무시 (rx_done · echo 0회), 뒤 byte 정상 1회	23 / 23
5	**Data bit glitch** 부록	0x55 · 0xAA, bit 0 ~ 7에 20 ns ~ 6 µs 반대 레벨 pulse, bit 중앙 ±0 · 1 · 7 tick	rx_data, tx echo, rx_done 횟수	1 tick 미만 glitch → data 유지	256 / 320

05

RX 타이밍: rx Fall 다음 b_tick에 START, 8 tick 뒤 DATA

결론

- ① rx deassert → b_tick = 1인 clk에 IDLE → START
- ② Sampling timing 일치 — START 52.08 µs · DATA bit 간격 104.16 µs

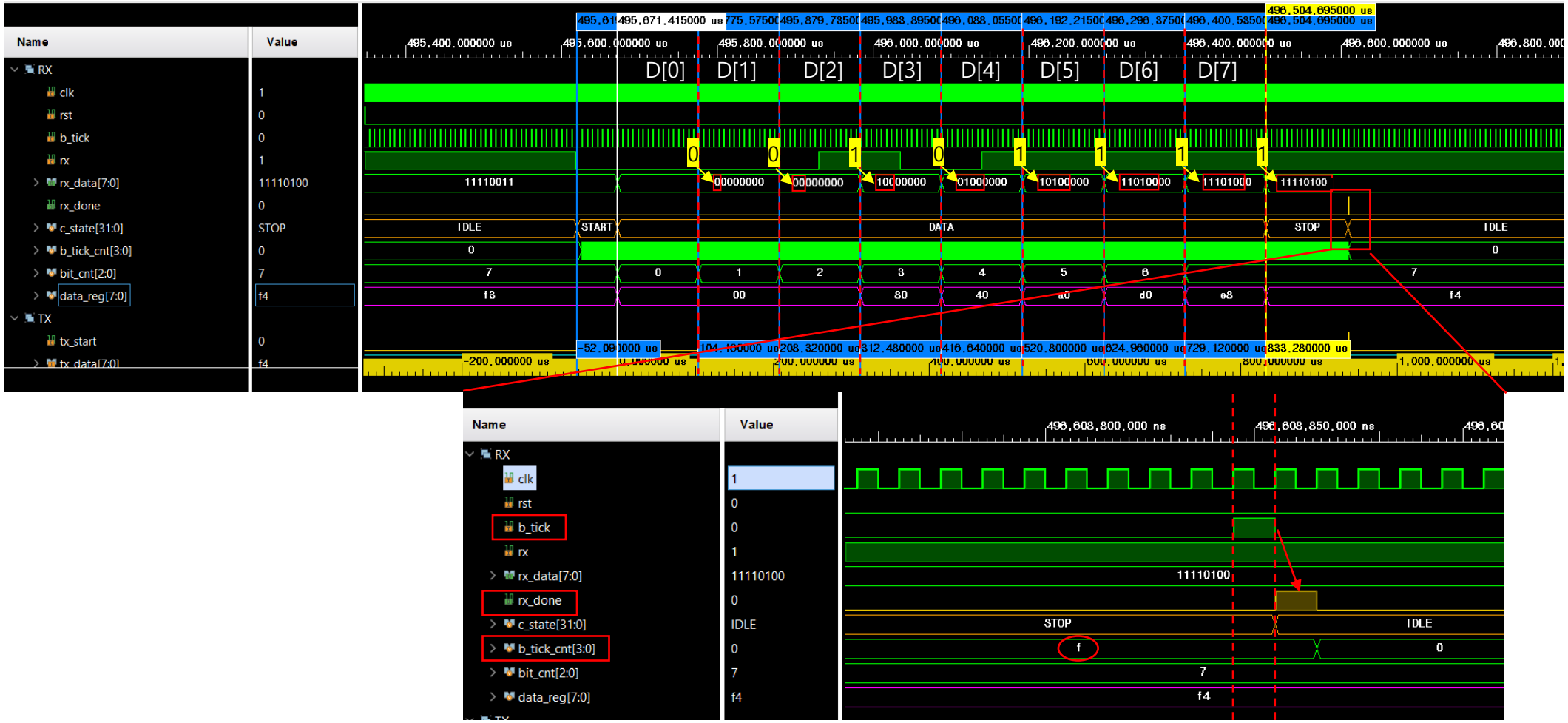


05

Data Sampling: b_tick_cnt == 15마다 1 bit Shift, STOP에서 8 bit 완성

결론

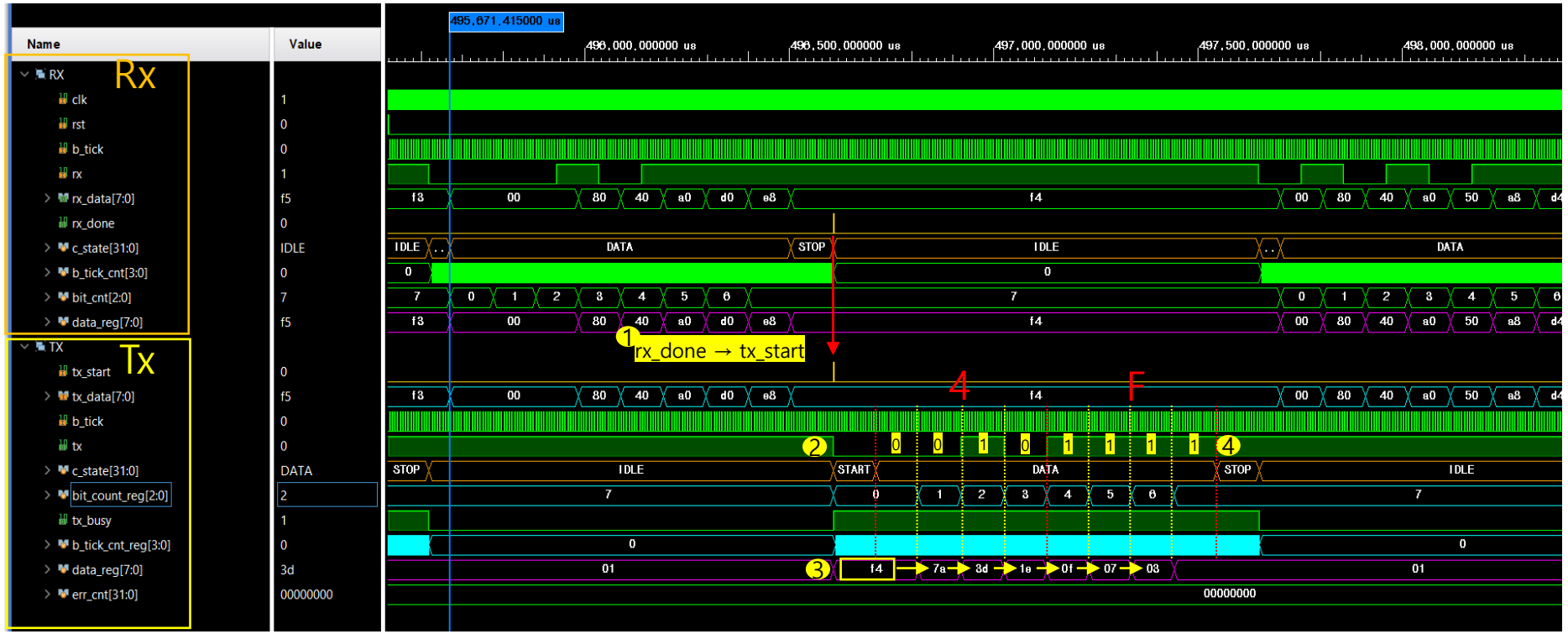
- DATA 구간 b_tick_cnt == 15마다 rx를 data_reg MSB에 넣고 오른쪽 shift → D0 ~ D7 수신
- STOP 중앙 (b_tick_cnt == 15)에서 rx_done 1 clk → ASM STOP 동작과 일치



TX Echo: rx_done → tx_start, 수신 Byte를 LSB부터 재송신

05

결론 · rx_done 1 clk → tx_start → tx가 같은 byte를 START · D0 ~ D7 · STOP으로 송신
· 8'h00 ~ 8'hFF 256 case echo 모두 PASS




```
INPUT for 256 case Result
        256 case  pass :         256  fail :          0
```

1 rx_done → tx_start
STOP 중앙 1 clk pulse

2 tx = 0 (START)
tx_start 다음 clk부터

3 data shift 송신
START에서 rx_data 저장
→ 16 tick마다 1 bit

4 tx = 1 (STOP)
STOP 후 IDLE 복귀

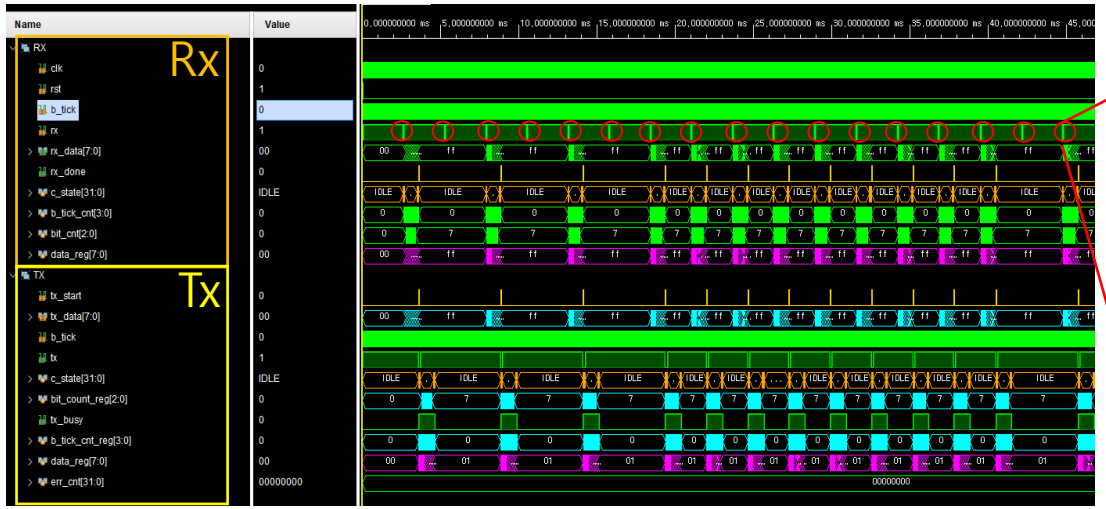
256 / 256 PASS

트러블 슈팅 1: rx Glitch가 가짜 Byte로 수신 · Echo

결론

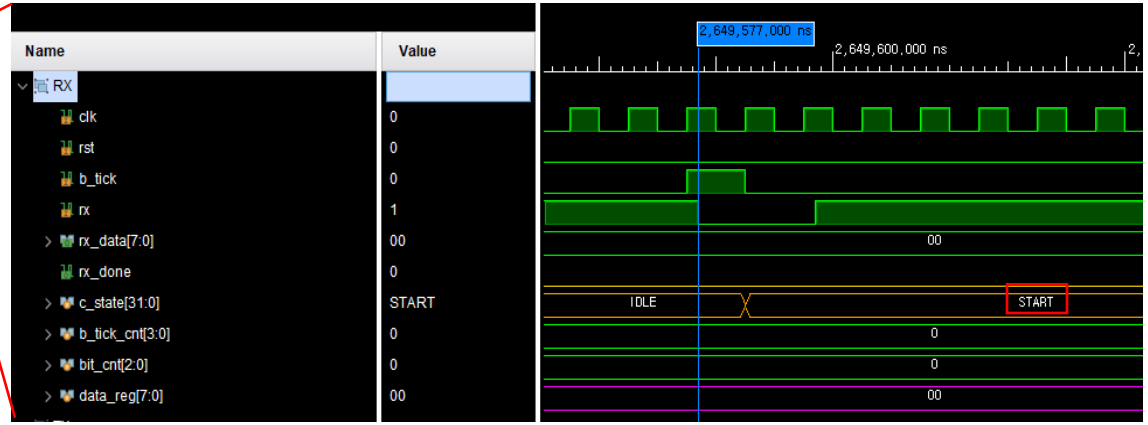
- **① 현상 — START glitch 23 case 중 17 FAIL, glitch 하나에 가짜 0xFF 수신 · echo**
- **② 원인 — START에서 rx를 다시 보지 않고 8 tick 뒤 DATA 진입 → glitch를 start bit로 처리**

① 현상 glitch test FAIL · Rx / Tx에 가짜 frame




```
---- glitch burst / chattering (expect: ignored) ----
44774288 ns : PASS glitch w=200 ns x5 gap=1000 off-tick : ignored
47394798 ns : FAIL glitch w=1000 ns x8 gap=1000 off-tick : rx_done 1 (last ff), tx echo 1 (last ff)
---- glitch then real byte (expect: byte received once) ----
51149288 ns : PASS glitch w=200 ns off-tick + 104160 ns + data 55 : received 55
54938007 ns : FAIL glitch w=200 ns on-tick  + 104160 ns + data 55 : rx_done 2 (last ff), tx echo 1 (last aa)
58726827 ns : FAIL glitch w=200 ns on-tick  + 104160 ns + data d5 : rx_done 1 (last aa), tx echo 1 (last aa)
62723967 ns : FAIL glitch w=200 ns on-tick  + 312480 ns + data 55 : rx_done 2 (last fe), tx echo 1 (last ab)
67658547 ns : FAIL glitch w=200 ns on-tick  + 1249920 ns + data 55 : rx_done 2 (last 55), tx echo 2 (last 55)
GLITCH Result  pass : 6  fail : 17
```

② 원인 IDLE → START 직후 rx 재확인 없음



트러블 슈팅 1: START에서 rx 재확인 → Glitch 23 Case PASS

결론

- ③ 해결 — START 동안 rx = 0일 때만 b_tick_cnt 증가, rx = 1일 경우 IDLE 복귀
- ④ 적용 결과 — glitch 23 case 모두 PASS, glitch 입력에도 tx 변화 없음

③ 해결 START 블록 수정 (전 → 후)

```
START: begin
    if (b_tick) begin
        if (b_tick_cnt == 7) begin
            b_tick_cnt_next = 0;
            bit_cnt_next    = 0;
            data_next       = 8'h00;
            n_state         = DATA;
        end else begin
            b_tick_cnt_next = b_tick_cnt + 1;
        end
    end
end
```

→

```
START: begin
  if(!rx)begin
    if (b_tick) begin

        if (b_tick_cnt == 7 ) begin
            b_tick_cnt_next = 0;
            bit_cnt_next    = 0;
            data_next       = 8'h00;
            n_state         = DATA;
        end

        else begin
            b_tick_cnt_next = b_tick_cnt + 1;
        end

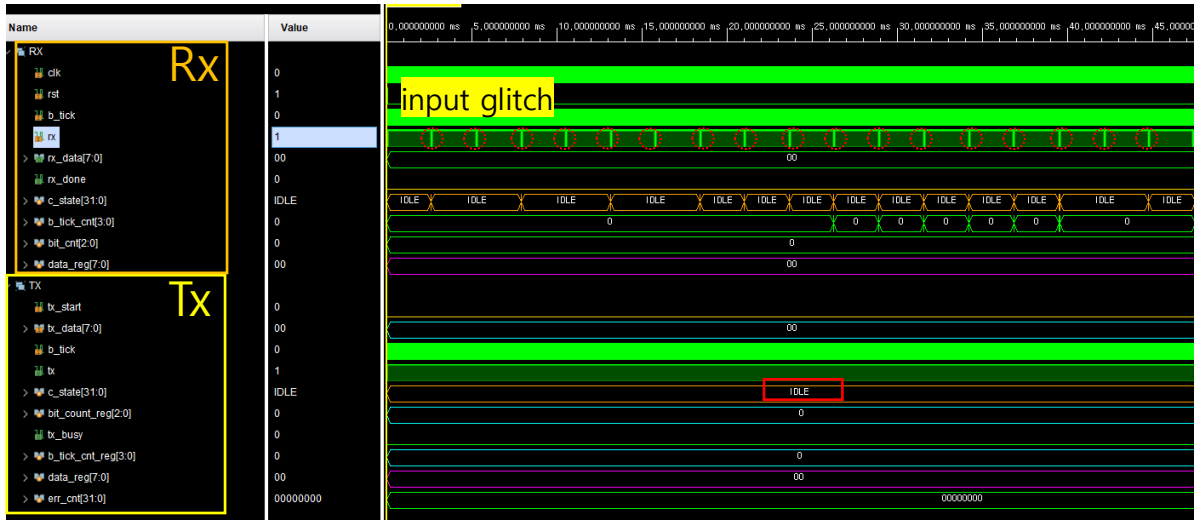
    end
  end
  else begin
            b_tick_cnt_next = 0;
            bit_cnt_next    = 0;
            data_next       = 8'h00;
            n_state         = IDLE;

  end

end
```

b_tick_count <7 상황에서 rx가 1이면 프로토콜 오동작이므로 다시 IDLE 상태로 전환

④ 적용 결과 GLITCH Result pass 23 · fail 0



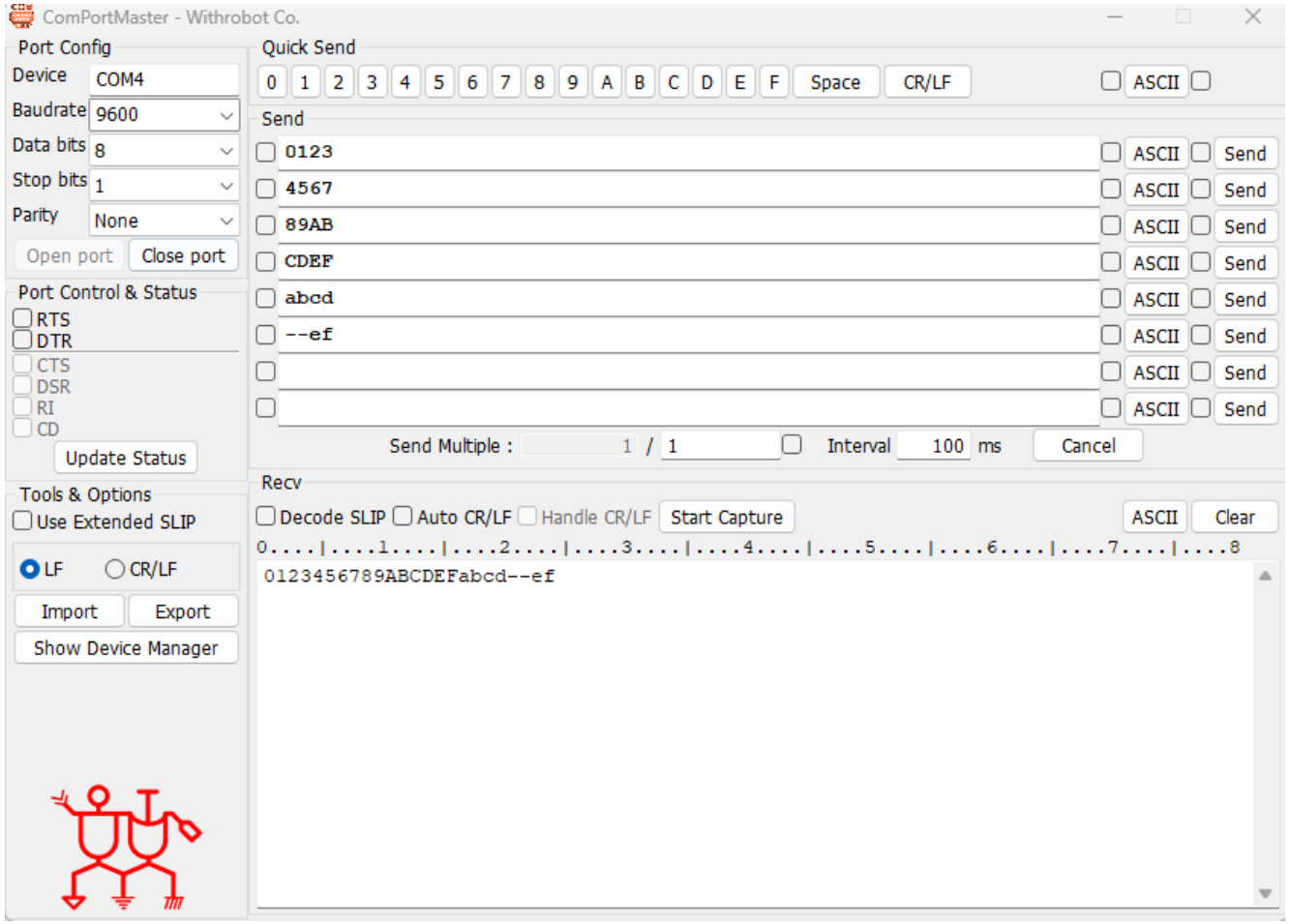

```
39500618 ns : PASS glitch w=50000 ns x1 gap=0 off-tick : ignored
42164247 ns : PASS glitch w=50000 ns x1 gap=0 on-tick  : ignored
---- glitch burst / chattering (expect: ignored) ----
44774288 ns : PASS glitch w=200 ns x5 gap=1000 off-tick : ignored
47394798 ns : PASS glitch w=1000 ns x8 gap=1000 off-tick : ignored
---- glitch then real byte (expect: byte received once) ----
51149288 ns : PASS glitch w=200 ns off-tick + 104160 ns + data 55 : received 55
54938007 ns : PASS glitch w=200 ns on-tick  + 104160 ns + data 55 : received 55
58726827 ns : PASS glitch w=200 ns on-tick  + 104160 ns + data d5 : received d5
62723967 ns : PASS glitch w=200 ns on-tick  + 312480 ns + data 55 : received 55
67658547 ns : PASS glitch w=200 ns on-tick  + 1249920 ns + data 55 : received 55
GLITCH Result  pass : 23  fail : 0
```

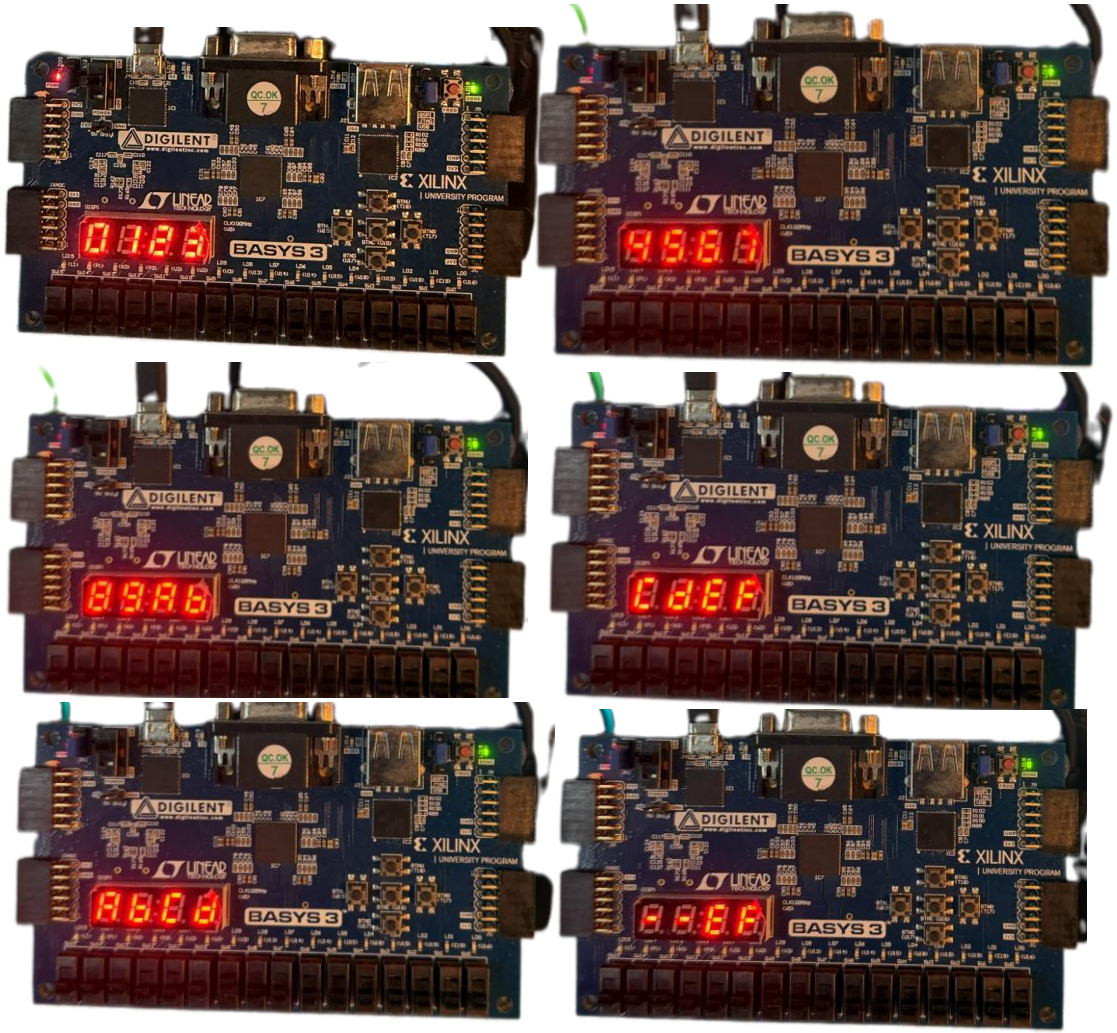
시연: PC 입력 6줄 모두 FND 표시 · Echo 일치

07

결론

- **ComPortMaster (COM4 · 9600 · 8N1)로 0123 · 4567 · 89AB · CDEF · abcd · --ef 송신 → FND에 마지막 4글자**
- **Recv 창 echo = 송신 문자열, 소문자 a ~ f도 hex 표시, hex 외 문자 '-'**





08

UART Loopback 구조로 uart_tx · uart_rx 검증 완료

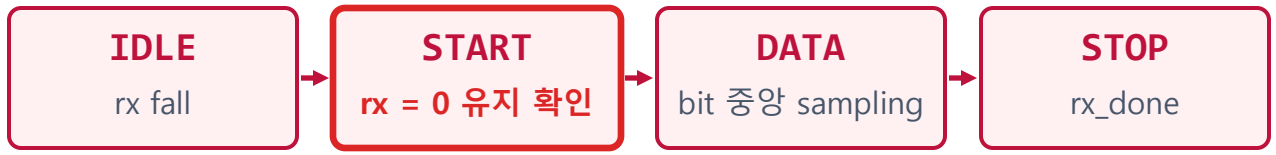
결론

- **Loopback echo 256 / 256 · 보드 시연 일치 → uart_tx · uart_rx 동작 검증 완료**
- **START에서 rx 재확인 → rx glitch로 인한 데이터 오입력 방지 (glitch 23 case 6 → 23 PASS)**

결론

검증	결과	판정
#1 RX 타이밍	ASM과 일치	PASS
#2 · #3 256 byte loopback echo	256 / 256	PASS
#4 START glitch (20 ns ~ 50 µs)	6 → 23 / 23	PASS
#5 Data bit glitch (1 tick 미만)	256 / 320	FAIL
보드 시연 (6줄 입력)	FND · echo 일치	PASS

데이터 오입력 방지 구조



고찰 — ONE BIT mode 한계

- bit 중앙 sample 1개로 판정 → DATA 구간 glitch가 그 sampling edge를 덮을 경우 해당 bit 반전
- 1 tick 미만 glitch 320 case 중 bit 중앙 64 case FAIL, ±1 · ±7 tick 위치는 PASS
- rx_done · echo는 정상으로 나가서 오류가 드러나지 않음

개선 방향

1. bit 중앙 3 sample (7 · 8 · 9 tick) 다수결 → 1 tick 미만 glitch는 1표만 바꿔 제거
2. 1 tick 이상 glitch → parity · CRC로 오류 검출 후 재전송 요청, 또는 ECC로 정정
3. TX 앞 FIFO + tx_busy 사용 → 연속 수신 시 echo 누락 방지

Q & A

감사합니다

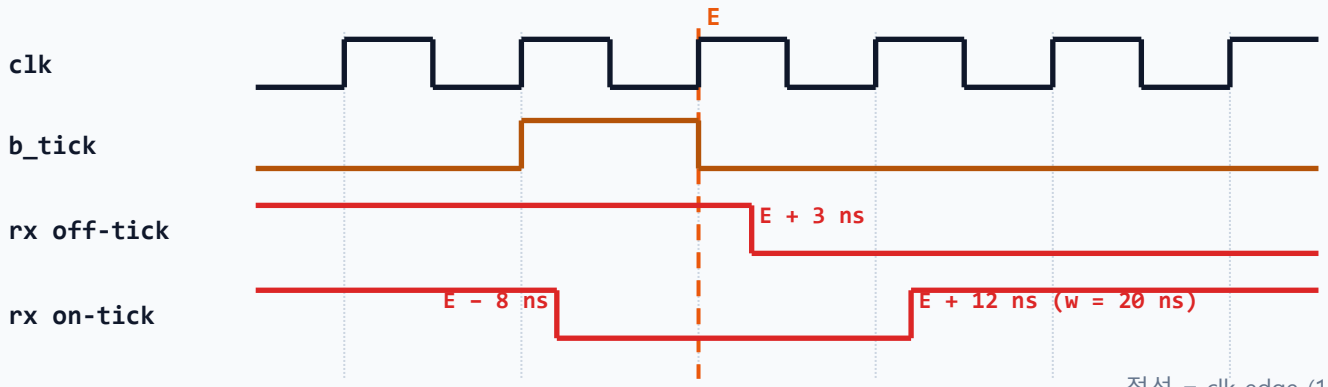
정안성

A

Glitch 입력: RX Sampling Edge 기준 On-tick / Off-tick 배치

결론 · IDLE은 b_tick = 1인 clk edge(E)에서만 rx 확인 → glitch 결과는 폭보다 **E를 덮었는지**로 결정
· off-tick = E + 3 ns 시작 (안 덮음), on-tick = E 중심 배치 (덮음) → 확률의 양 끝 0 % · 100 % 확인

ns 확대 · sampling edge E 주변



µs 단위 · E 간격 1 tick = 6,510 ns



입력 task

align_glitch
b_tick 상승 후 지연으로 E 기준 위치 고정

sender_glitch (폭, 개수, 간격)
low pulse 개수만큼 반복 → 종료 후 rx = 1

check_glitch
rx_done · tx echo 횟수 비교 → PASS / FAIL

1. `off-tick` E + 3 ns에서 low 시작
   폭 6,507 ns 미만 → 다음 E 안 덮음

2. `on-tick` E 중심 + 2 ns에 low 배치
   폭과 무관하게 E 덮음

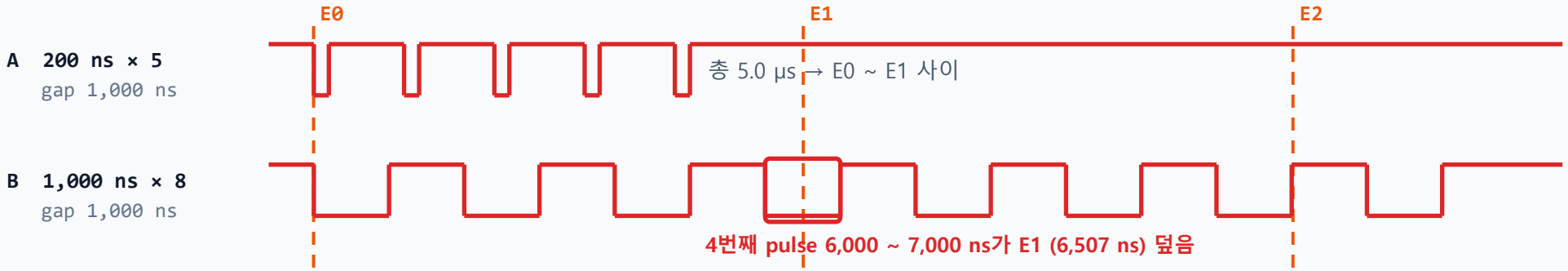
3. `+2 / +3 ns` rx edge를 clk 격자 밖에 둠
   TB · RTL race 없음

A

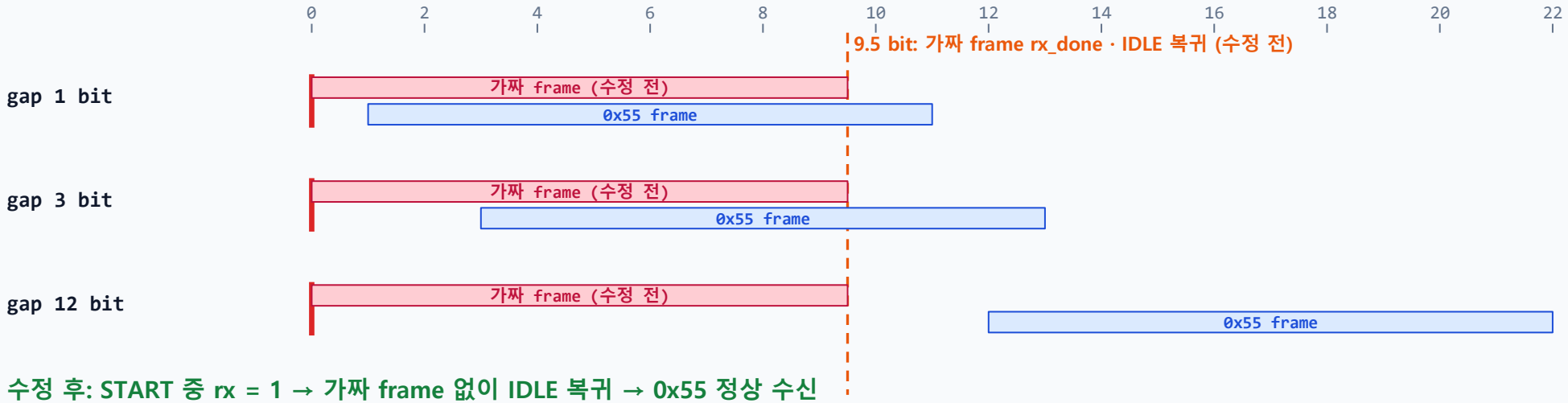
Burst · Glitch 뒤 Byte: Sampling Edge 통과와 Byte 손실 확인

결론
- **Burst — 짧은 pulse 여러 개가 E를 지나가는 chattering 재현 (5 µs · 15 µs)**
- **Glitch + byte — glitch 뒤 1 · 3 · 12 bit 간격으로 실제 byte 송신 → 가짜 frame과 겹침 확인**

Burst (chattering) · t = 0 = E0 + 3 ns



Glitch + byte · 200 ns glitch(on-tick) → gap → 0x55, 단위 bit (1 bit = 104,160 ns)



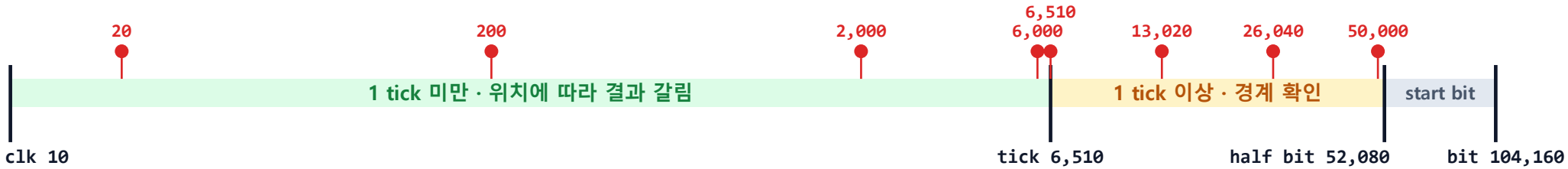
수정 후: START 중 rx = 1 → 가짜 frame 없이 IDLE 복귀 → 0x55 정상 수신

A

Glitch 폭: clk · tick · Half-bit 경계 기준으로 선정

결론 · 기준 시간 상수 3개 — clk 10 ns · tick 6,510 ns (sampling edge 간격) · half bit 52,080 ns (START 대기)
· 1 tick 미만 = 실제 노이즈 구간 (위치에 따라 결과 갈림), 1 tick 이상 = 경계 확인용

단위 ns · log scale



폭 (ns)	환산	선정 근거	구분
20	2 clk	RTL sim은 rx를 clk edge(10 ns)에서만 봄 → clk edge를 확실히 덮는 최소 폭	짧은 noise
200	20 clk	1 tick 안 10배 간격 sweep	짧은 noise
2,000	200 clk	1 tick 안 10배 간격 sweep	짧은 noise
6,000	1 tick − 510	off-tick이 E를 피하는 최대 폭 6,507 ns 바로 아래	짧은 noise
6,510	1 tick	sampling edge 간격과 같은 폭 → 모든 위치에서 E 1개 이상 덮음	경계 확인
13,020	2 tick	E 2개 이상 덮음	경계 확인
26,040	4 tick (1/4 bit)	START 대기 8 tick의 중간	경계 확인
50,000	half bit 미만	START 대기 8 tick(52,080 ns) 바로 아래 → START 재확인으로 거를 수 있는 상한	경계 확인

A

START Glitch: 수정 전 17 FAIL → 수정 후 23 Case 모두 PASS

결론
- 수정 전 — sampling edge를 덮은 glitch는 폭과 무관하게 가짜 0xFF 수신 · echo, 뒤 byte까지 손실
- 수정 후 — START 중 rx = 1 → IDLE 복귀, 50 µs glitch까지 무시 · 뒤 byte 정상 1회 수신

입력	폭 (ns) · 조건	위치	수정 전 rx_done / echo	수정 전	수정 후	case
single	20 · 200 · 2,000 · 6,000	off-tick	0 / 0	PASS	PASS	4
single	20 · 200 · 2,000 · 6,000	on-tick	1 (FF) / 1 (FF)	FAIL	PASS	4
single	6,510 · 13,020 · 26,040 · 50,000	off · on	1 (FF) / 1 (FF)	FAIL	PASS	8
burst	200 × 5, gap 1,000	off-tick	0 / 0	PASS	PASS	1
burst	1,000 × 8, gap 1,000	off-tick	1 (FF) / 1 (FF)	FAIL	PASS	1
glitch + byte	200 → 1 bit → 0x55	off-tick	1 (55) / 1 (55)	PASS	PASS	1
glitch + byte	200 → 1 bit → 0x55	on-tick	2 (last FF) / 1 (AA)	FAIL	PASS	1
glitch + byte	200 → 1 bit → 0xD5	on-tick	1 (AA) / 1 (AA)	FAIL	PASS	1
glitch + byte	200 → 3 bit → 0x55	on-tick	2 (last FE) / 1 (AB)	FAIL	PASS	1
glitch + byte	200 → 12 bit → 0x55	on-tick	2 (last 55) / 2	FAIL	PASS	1

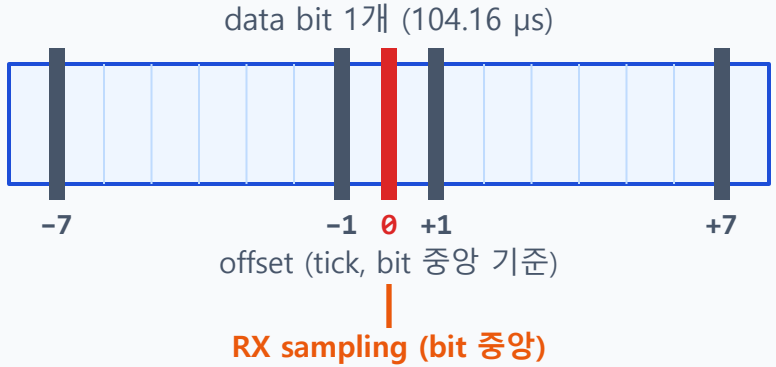
START Glitch Result · 수정 전 pass 6 / fail 17 → 수정 후 pass 23 / fail 0 (xsim, tb_uart.v)

Data Bit Glitch: Bit 중앙 Sampling Edge를 덮을 때만 1 bit 반전

A

결론 · 1 tick 미만 glitch 320 case 중 bit 중앙 (offset 0) 64 case FAIL — glitch 넣은 bit만 반전, rx_done · echo는 정상 1회
· ±1 · ±7 tick 위치는 모두 PASS → bit 중앙 1회 sampling (ONE BIT mode) 구조 한계

입력 방법




```
data  0x55 · 0xAA  (bit마다 0 · 1 둘 다)
bit   0 ~ 7 중 1개, 그 bit와 반대 레벨
폭    20 · 200 · 2,000 · 6,000 ns
판정  rx_data · echo = 송신 byte, 각 1회
```

FAIL 예 · 0x55, bit 0, 20 ns, offset 0

```
expected : 55 (01010101)
rx_data  : 54 (01010100)  rx_done x1
diff     :    (00000001)
```

PASS 수 / 16 (bit 8개 × data 2개)

폭 / offset	-7 tick	-1 tick	0 tick	+1 tick	+7 tick
20 ns	16 / 16	16 / 16	0 / 16	16 / 16	16 / 16
200 ns	16 / 16	16 / 16	0 / 16	16 / 16	16 / 16
2,000 ns	16 / 16	16 / 16	0 / 16	16 / 16	16 / 16
6,000 ns	16 / 16	16 / 16	0 / 16	16 / 16	16 / 16

DATA GLITCH Result · 320 case pass 256 / fail 64 (xsim, tb_uart.v)